Class 6: R Functions

Ryan Ellis (PID:A17673864)

Table of contents

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the functions)
- input *arguments* (one or more comma separated inputs that go inside brackets when we call the function)
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Lets call it `add()`.

Input arguments can be either *"required"* or *"optional"*. The latter have fall-back **default** values (if no Y value is inputted can default to 0, but must put into argument =0)

```
add <- function(x,y=0) {x + y}
```

Can we use our new function?

```
add(10,100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

For you second version, adapt your first function so it can take a single input vector or two inputs as before. For example, add(4, 7) and add(c(4,7,10)). [1 pt]

```
add <- function(x,y=0) {x + y}
```

```
add(4,7)
```

```
[1] 11
```

```
add(c(4,7,10))
```

```
[1]  4  7 10
```

c. Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, add(1, 2, 3, -4) [2pts]

```
add<- function (x,y,z){sum(x,y,z)}
```

```
add(1,2,3)
```

```
[1] 6
```

We can explicitly set **return** value input from a function (rather than the default last line) by using `return()` function call.

```
add<- function (x,y,z){
  returnsum(x,y,z)}+
  cat("is it break time yet?/n")
```

A generate_dna() function

a useful function here is the "base R" `sample()` function:

```
sample(1:5, size=3)
```

```
[1] 1 2 3
```

replace=true, will allow the same number or variable to be used over and over again

```
sample(1:5, size=5, replace=TRUE)
```

```
[1] 4 3 3 4 3
```

We can use this to make a random nucleotide sequence if we draw from "A", "G", "C" and "T" ...

```
sample(x=c("A","G","C","T"),size=10, replace=TRUE)
```

```
 [1] "G" "A" "T" "C" "G" "T" "T" "G" "G" "C"
```

> Q2:Write a function generate_dna() that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example generate_dna(6) might return "A", "T", "T", "G", "A",

```
generate_dna <- function(len=10){sample(x=c("A","G","C","T"),size=len, replace=TRUE)}
```

```
generate_dna(len=100)
```

```
  [1] "A" "C" "C" "G" "C" "A" "G" "T" "C" "T" "A" "T" "A" "T" "C" "C" "T" "C"
 [19] "C" "T" "C" "T" "G" "T" "G" "G" "G" "C" "C" "G" "T" "G" "A" "A" "C" "T"
 [37] "A" "T" "G" "A" "G" "A" "G" "A" "A" "A" "G" "A" "G" "A" "A" "G" "A" "A"
 [55] "C" "G" "G" "G" "A" "G" "T" "T" "C" "T" "G" "T" "C" "A" "G" "C" "C" "G"
 [73] "T" "G" "C" "G" "T" "C" "A" "C" "G" "A" "C" "A" "C" "C" "T" "G" "G" "A"
 [91] "C" "G" "C" "T" "A" "G" "T" "C" "C" "G"
```

```
generate_dna()
```

```
 [1] "C" "C" "G" "T" "A" "G" "T" "G" "C" "T"
```

```
#generate_dna(len=100)
```

> Q2b: Your second version should *optionally* be able to return either a multi-element vector of single character nucleotides (as before) or a **single character string** (not a vector of single letters but a single vector of multiple letters). For example "AAGGCTG". [1 pt]

```
generate_dna <- function(len=10){sample(x=c("A","G","C","T"),size=len, replace=TRUE)}
```

functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
generate_dna <- function(len=10,single.element=TRUE){ans<-sample(x=c("A","G","C","T"),size=le
paste (ans,collapse="")
if(single.element){ans<- paste(ans, collapse="")}


##Format as FASTA with and ID line

cat(
  paste(">len", len, "\n", sep="")
  )
cat(ans)
cat("\n")}
```

```
generate_dna(len=10, single.element =FALSE)
```

```
>len10
G C A T T G G A A A
```

```
generate_dna(4, T)
```

```
>len4
AGAA
```

```
paste( c("A", "C", "G"), collapse= "")
```

```
[1] "ACG"
```

Q2c. Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
>len9
CGAAGGCTG
```

```
cat("hello \n there")
```

```
hello
 there
```

##Write a `generate_protien()` function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return `"WQRTAG"`.

```
generate_protien<- function(len=9){
  aa<-c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T",

ans<- sample(x=aa, size=len, replace=TRUE)
paste (ans,collapse="")
}
```

```
generate_protien(4)
```

```
[1] "YEAG"
```

Generate random protien sequence

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function for() or sapply() so that you do not have to call generate_protein() eight times by hand.

```r
for(l in 6:13) {
  cat(">",l, "\n", sep="")
  cat(generate_protien(l), "\n")
}
```

```
>6
QYVLSC
>7
VGYSNGI
>8
NKFAAVEK
>9
RDYIVNPYQ
>10
YTLTAECLIV
>11
VGATNMICFRG
>12
GKWVSAYQKAAA
>13
HWKHNNIPGKTEI
```

Are out peptides "unique in nature"?

> Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at https://blast.ncbi.nlm.nih.gov/. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).
>
> **Table** was created differently than in class, Instead of copying and pasting I used a `data.frame()`. My file didnt save in class, and the best way to create a table found through triton GPT was constructing a `data.frame`, wiht `knitr` (*I could not remember how to insert a table into the r console*)

```r
my_table <- data.frame(
  "Length(AA)" = c("6", "7", "8", "9", "10", "11", "12", "13"),
  "BestHitIdentity" = c(100, 100, 100, 100, 89, 80, 73, 75 ),
  "BestHitCoverage" = c(100, 100, 88, 78, 90, 91,92, 92),
  "Unique y/n" = c("n", "n","y", "y", "y", "y", "y", "y")
)

knitr::kable(my_table)
```

Length.AA.	BestHitIdentity	BestHitCoverage	Unique.y.n
6	100	100	n
7	100	100	n
8	100	88	y
9	100	78	y
10	89	90	y
11	80	91	y
12	73	92	y
13	75	92	y

A) At which sequence length do your randomly generated peptides start to look "unique in nature" (i.e. no 100% coverage + 100% identity hit)? [1 pt]

At sequence length 8 is when the sequeneces started to all be random, with both the percent idenitity and query coverage dropping to below 90%

B) Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20 for a peptide of length) and to the size of the known protein universe. [1 pt]

The known amino acid universe is around 20 amino acids, to get a unique amino acid sequence using just 5 or 6 is unlikely due to the number of combinations you can make with just 6 AA. its is still a large number but not impossible. For instance for a 6 amino acid sequence there are 20^6 possible combinations. While the database is vast you could still find a unique sequence. But as the sequence length increases so will the number os possibilities till your left with mostly unique sequences.

Q6. Connecting your findings to immunology (MHC class II and T-cell

activation)

A key biological observation is that MHC class II molecules preferentially bind peptides that are a certain minimum length (peptides shorter than this are not used for presentation).In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

Because as we have discussed above short amino acid sequences are less likely to be unique. So if MHC is recognizing these short sequences, then by chance it is likely that it will recognize and bind a non unique sequence . this would lead to the antigen or infectious cell evading an immune response and likely lead to sickness of the host.